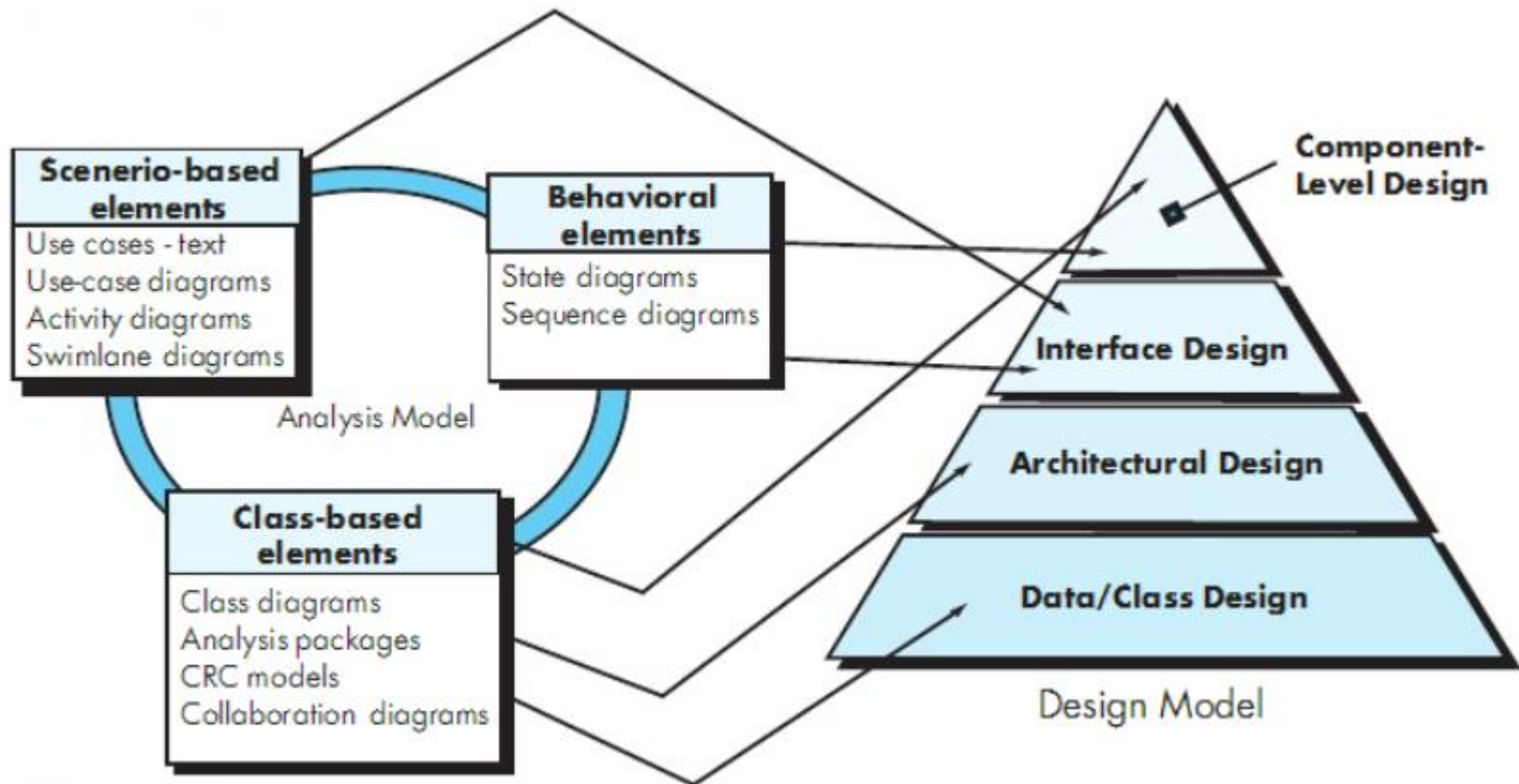


Designing System Architecture

The Design Process

- **Design** is the creative process of figuring out how to implement all of the customer's requirements; the resulting plan is also called the design
- Early design decisions address the system's architecture
- Later design decisions address how to implement the individual units

Translating Requirements to Design



The Design Process

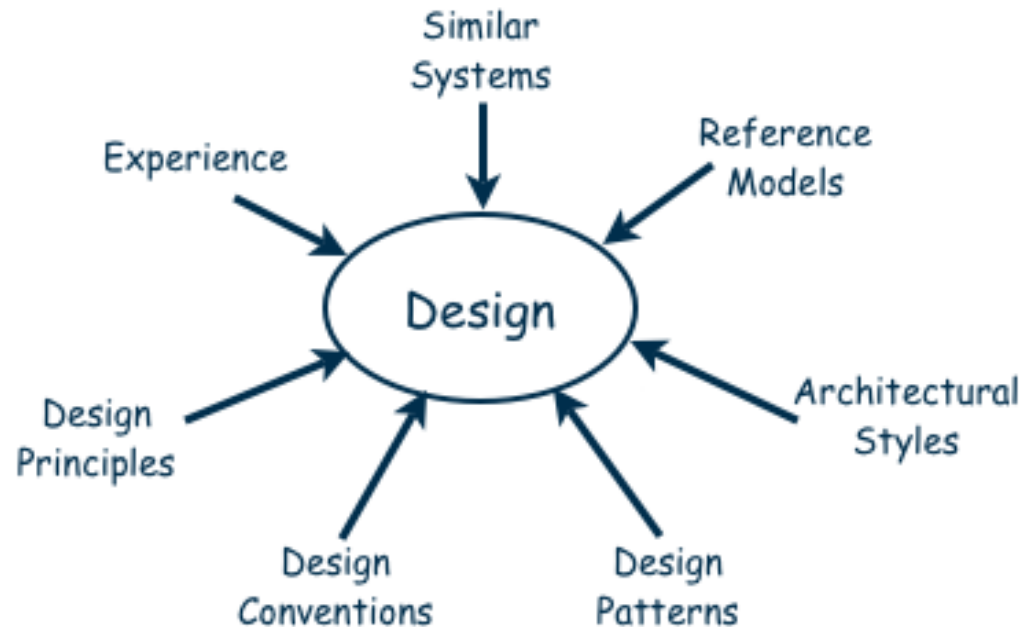
Design is a Creative Process

- Design is an intellectually challenging task
 - Numerous possibilities the system must accommodate
 - Non functional design goals (e.g., ease of use, ease to maintain)
 - External factors (e.g government regulations)
- We can improve our design by studying examples of good design
- Most design work is **routine design**, solve problem by reusing and adapting solutions from similar problems

The Design Process

Design is a Creative Process (continued)

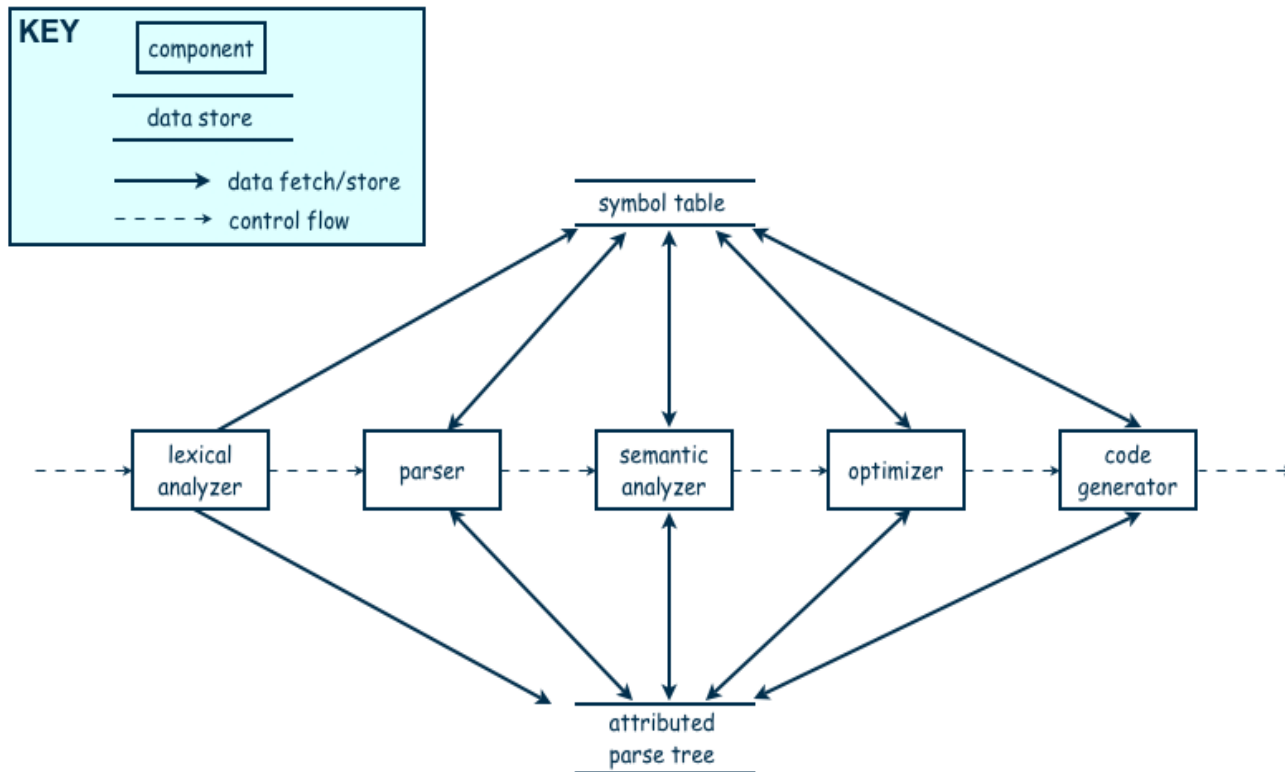
- Many ways to leverage existing solutions
 - Cloning: Borrow design/code in its entirety, with minor adjustments
 - Reference models: Generic architecture that suggests how to decompose the system



The Design Process

Design is a Creative Process (continued)

- Reference model for a compiler



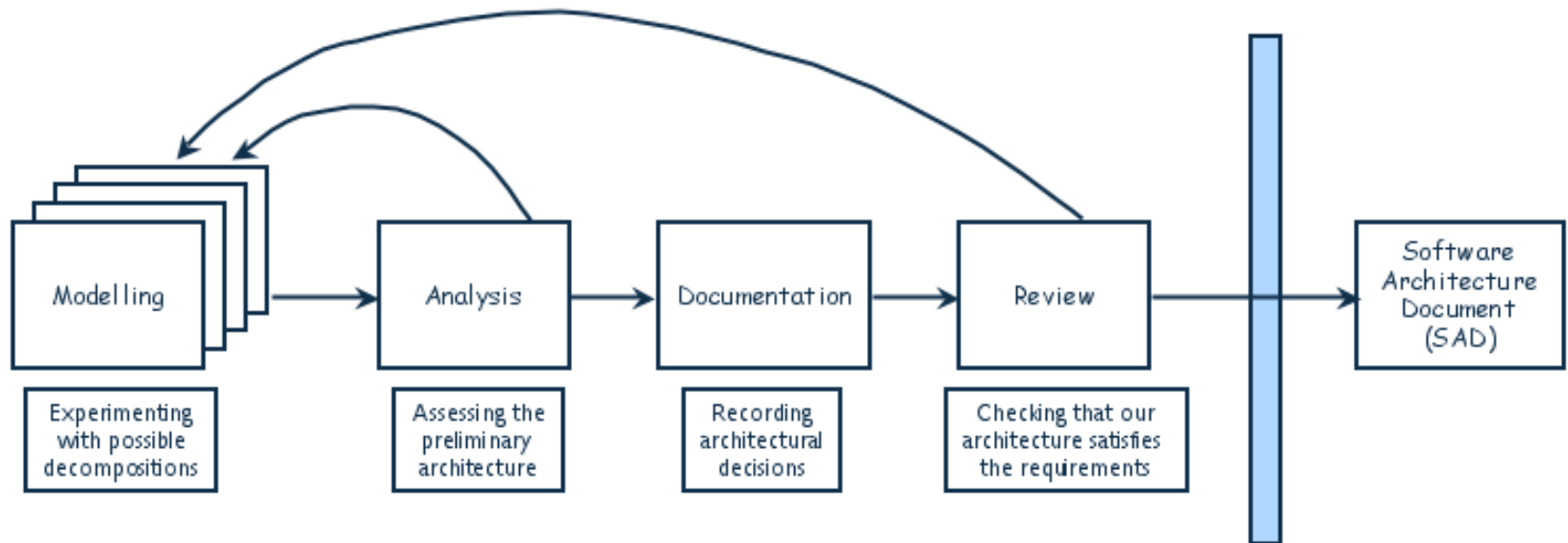
The Design Process

Design is a Creative Process (continued)

- More typically, a reference model will not exist for the problem
- Software architectures have generic solutions too, referred to as **architectural styles**
 - Focusing on one architectural style can create problems
 - Good design is about selecting, adapting, and integrating several architectural design styles to produce the desired result

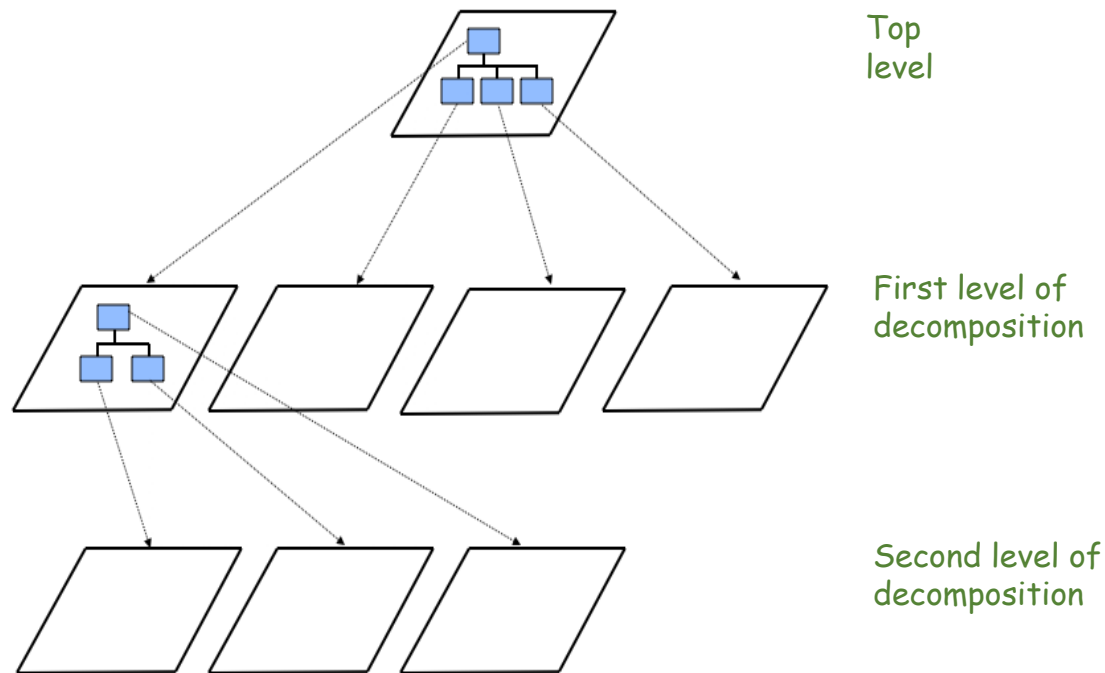
The Design Process

- The designers move back and forth among activities involving:
 - Understanding requirements, proposing solutions, testing feasibility of a solution, presenting possibilities to the customers, documenting the design for programmers
- The final outcome is the software architecture document (SAD)



Decomposition and Views

- High-level description of system's key elements
- Creating a hierarchy of information with increasing details



Decomposition and Views

Popular Design Methods

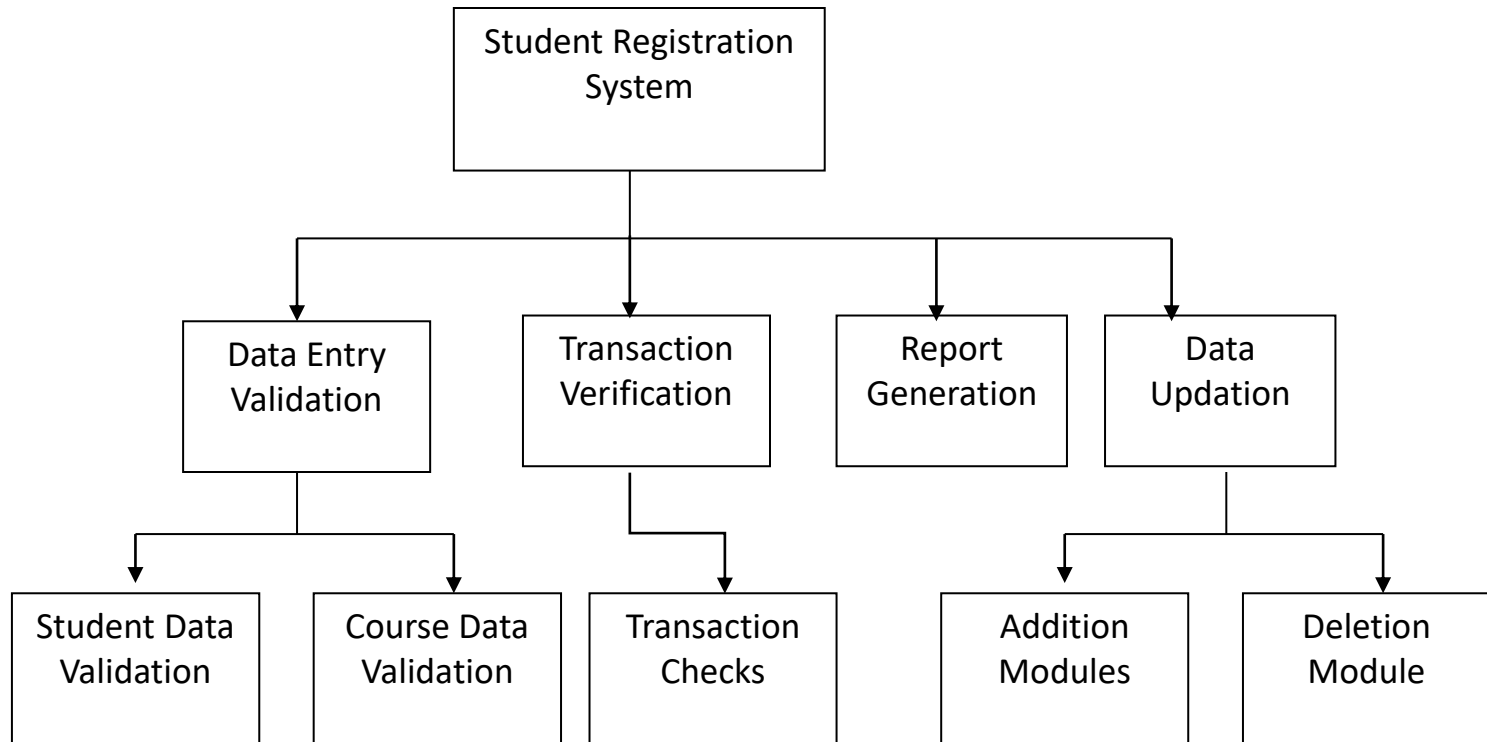
- Some design problems have no existing solutions
 - Designers must decompose to isolate key problems
- Some popular design methods:
 - Functional decomposition
 - Feature-oriented decomposition
 - Data-oriented decomposition
 - Event-oriented decomposition
 - Object-oriented design

Decomposition and Views

Popular Design Methods

- **Functional decomposition**
 - partitions functions or requirements into modules
 - begins with the functions that are listed in the requirements specification
 - lower-level designs divide these functions into subfunctions, which are then assigned to smaller modules
 - describes which modules (subfunctions) call each other

Functional Decomposition Example



Decomposition and Views

Popular Design Methods (continued)

- A design is **modular** when each activity of the system is performed by exactly one software unit, and when the inputs and outputs of each software unit are well-defined

Design Methodology

- We have an abstract description of a solution to our customer's problem, a software architectural design, a plan for decomposing the design into software units and allocating the system's functional requirements to them
- No distinct boundary between the end of the architecture-design phase and the start of the module-design phase
- No comparable design recipes for progressing from a software unit's specification to its modular design
- The process taken towards a final solution is not as important as the documentation produced

Design Methodology

- Design decisions are periodically revisited and revised
- Refactoring
 - to simplify complicated solutions or to optimize the design

Design Documentation

- The details of the system architecture is documented in *Software Architecture Document* (SAD)
- SAD serves as a bridge between the requirements and the design
- Detailed (or module) design acts as a bridge from architecture design to code
- Many ways to document the design
- **Design by contract:** a particular approach that uses the documentation not only to capture the design but also to encourage interaction among developers

Architectural Styles and Strategies

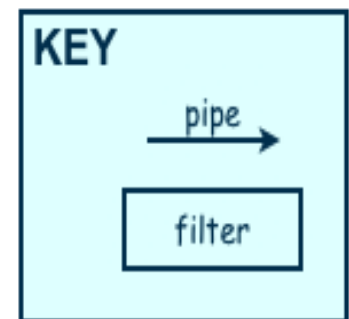
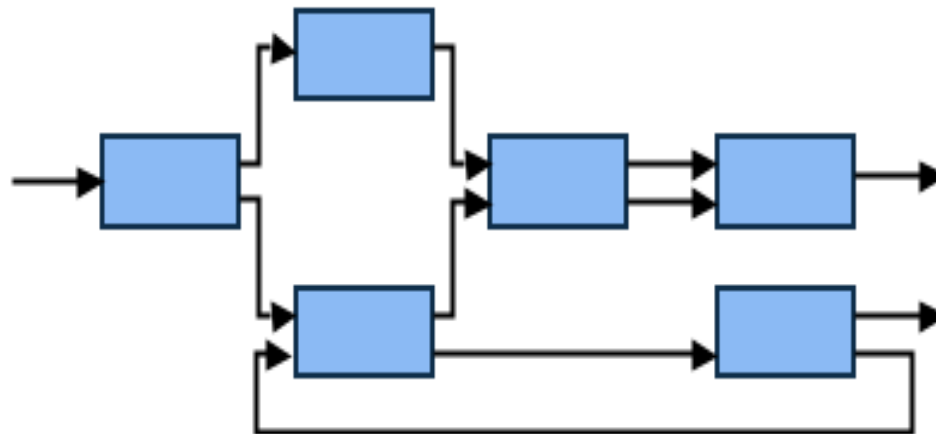
- Pipe-and-Filter
- Client-Server
- Peer-to-Peer
- Publish-Subscribe
- Repositories
- Layering

Architectural Styles and Strategies

Pipe-and-Filter

- The system has
 - Streams of data (pipe) for input and output
 - Transformation of the data (filter)
 - Filters process the data, and Pipes move the data between filters.
- Think of a **water treatment plant**:

Water enters → passes through **filters for sediment** → then **chlorination filter** → then **final clean water output**.



Architectural Styles and Strategies

Pipe-and-Filter (continued)

- **Several important properties**
 - The designer can understand the entire system's effect on input and output as the composition of the filters
 - The filters can be reused easily on other systems
 - System evolution is simple
 - Allow concurrent execution of filters

Pipe-and-Filter

- **Use Cases:** Image Processing: Filters in a pipeline transform images step by step, applying effects like blurring or color adjustments.
- **Text processing and compilation:** Compiler stages: Lexical analysis → Syntax → Semantic → Code generation
- **Audio Signal Processing:** Filters modify audio signals in sequence, such as noise reduction or equalization.

Advantages

Benefit	Explanation
Modularity	Filters are independent and easy to understand/test
Reusability	Filters can be reused in different pipelines
Scalability	Filters can be added, removed, or replaced without affecting others
Parallelism	Filters can run concurrently if the data flow allows
Maintainability	Easier to update one filter without affecting the rest
Extensibility	New transformations can be added by adding new filters

Disadvantages

Drawback

Explanation

Performance Overhead

Passing data between filters may add latency, especially if pipes are implemented as queues

Error Propagation

Errors in one filter can affect the whole pipeline if not handled

Debugging Difficulty

Tracing data through multiple filters can be harder

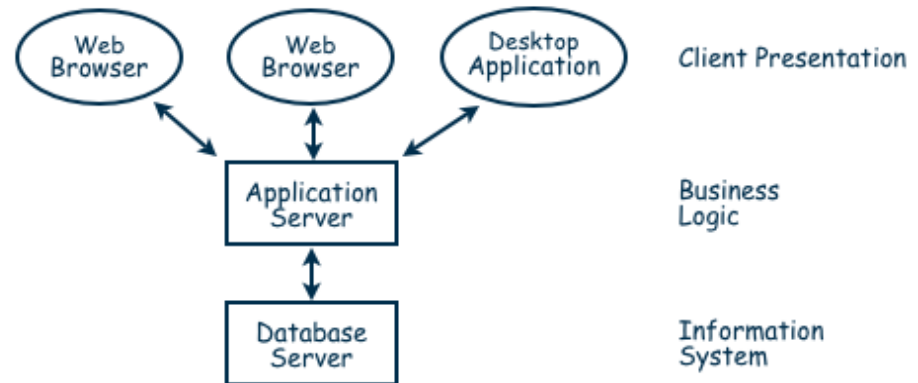
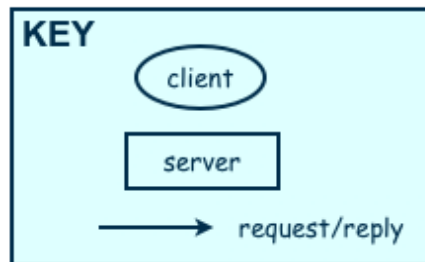
Not Ideal for Real-Time Systems

High-latency pipes may not work well for interactive systems

Architectural Styles and Strategies

Client-Server

- Two types of components:
 - Server components offer services
 - Clients access them using a request/reply protocol
- Client may send the server an executable function, called a callback
 - The server subsequently calls under specific circumstances



Client Server

- **Use cases:** Email System: Allows clients to send requests for retrieving or sending emails to a central server.
- Online Gaming: Clients interact with a central server for real-time game updates and multiplayer interactions.
- Remote File Storage: Clients access a server to store and retrieve files securely from a remote location.

Advantages

Advantage	Explanation
Centralized data	All data stored and managed on servers
Easy maintenance	Updates occur on server side
Scalability	Servers can be upgraded or replicated
Security	Access control can be centralized
Resource sharing	Multiple clients use the same services

Disadvantages

Drawback	Explanation
Server dependency	If server fails, services stop
Network dependency	Requires stable network connection
Server overload	Too many clients may overload server
Cost	High-performance servers can be expensive
Security risks	Central server may be targeted by attacks

Architectural Styles and Strategies

Peer-to-Peer (P2P)

- Each component acts as its own process and acts as both a client and a server to other peer components.
- Any component can initiate a request to any other peer component.
- Characteristics
 - Increased system capabilities
 - Highly tolerant of failures
- Examples: Napster, Kaaza, BitTorrent

Architectural Styles and Strategies

Peer-to-Peer (P2P)

P2P is useful when:

- Systems require **large-scale distributed resources**
- Centralized servers would become **bottlenecks**
- Systems need **high fault tolerance**
- Many nodes can contribute resources
- When NOT to use:
 - When file contents change frequently (e.g. prices)
 - When sharing speed has importance (e.g. large files are needed quickly)
 - File quality is critical
 - When trust between peers is required (e.g. the content is protected)

Architectural Styles and Strategies

Peer-to-Peer (P2P)

- **Use Cases:**

- Decentralized File Sharing: Users share files directly with each other without a central repository.
- Blockchain Networks: Distributed ledgers where each node maintains a copy of the entire blockchain.
- Collaborative Tools: Peer-to-peer architecture allows direct sharing of resources in collaborative applications.

- **Drawbacks:**

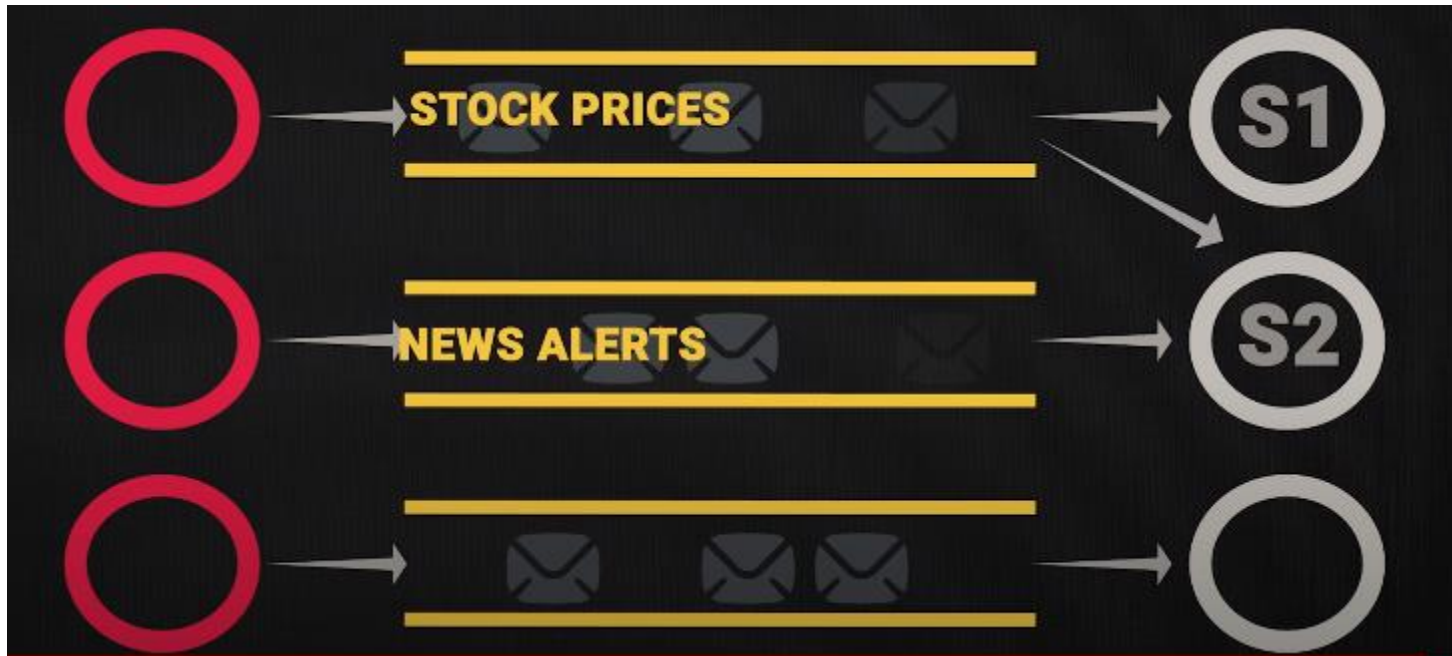
- Security concerns due to direct connections between nodes.
- Lack of central control can lead to coordination issues.

Architectural Styles and Strategies

Publish-Subscribe

- Components interact by broadcasting and reacting to events
 - Component expresses interest in an event by subscribing to it
 - When another component announces (publishes) that event has taken place, subscribing components are notified
 - Implicit invocation is a common form of publish-subscribe architecture
 - Registering: subscribing component associates one of its procedures with each event of interest (called the procedure)
- Characteristics
 - Strong support for evolution and customization
 - Easy to reuse components in other event-driven systems
 - Need shared repository for components to share persistent data
 - Difficult to test

Publish-Subscribe



weather station

building site

ships

tourists

Publish-Subscribe

Use Case: Event-Driven Systems: Applications that are event-driven, where actions or changes trigger events that need to be handled by different components, benefit from pub-sub architecture. For example, a notification system that sends alerts to users based on specific events, such as new messages, updates, or system status changes, can use this pattern.

Architectural Styles and Strategies

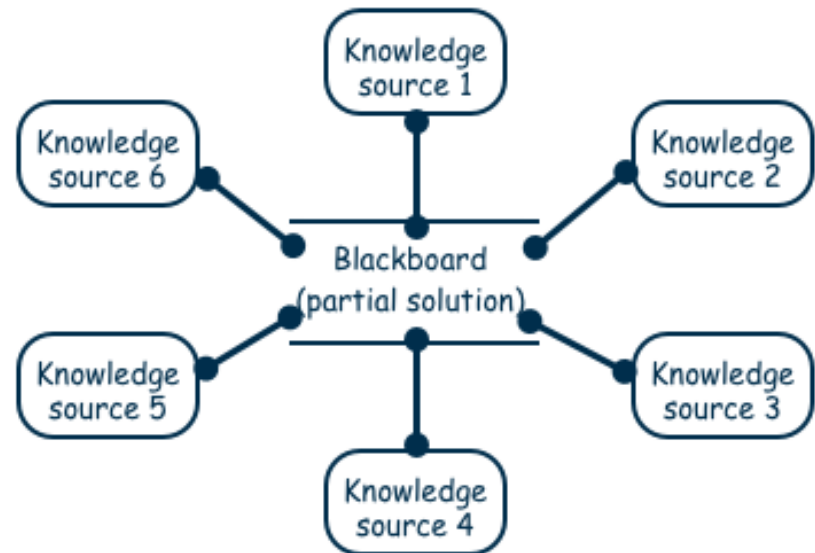
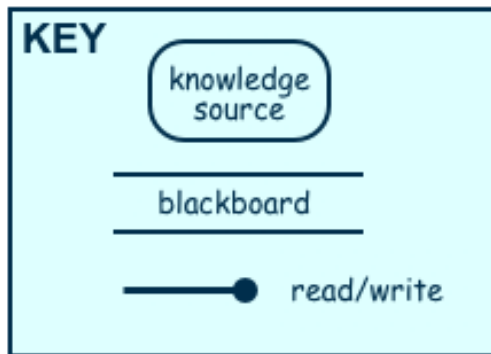
Repositories

- Two components
 - A central data store
 - A collection of components that operate on it to store, retrieve, and update information (data accessors)
- The challenge is deciding how the components will interact
 - A traditional database: transactions trigger process execution
 - A blackboard: the central store controls the triggering process
 - Knowledge sources: information about the current state of the system's execution that triggers the execution of individual data accessors

Architectural Styles and Strategies

Repositories (continued)

- Openness?
 - Data representation is made available to various programmers (vendors) so they can build tools to access the repository
 - But also a disadvantage: the data format must be acceptable to all components



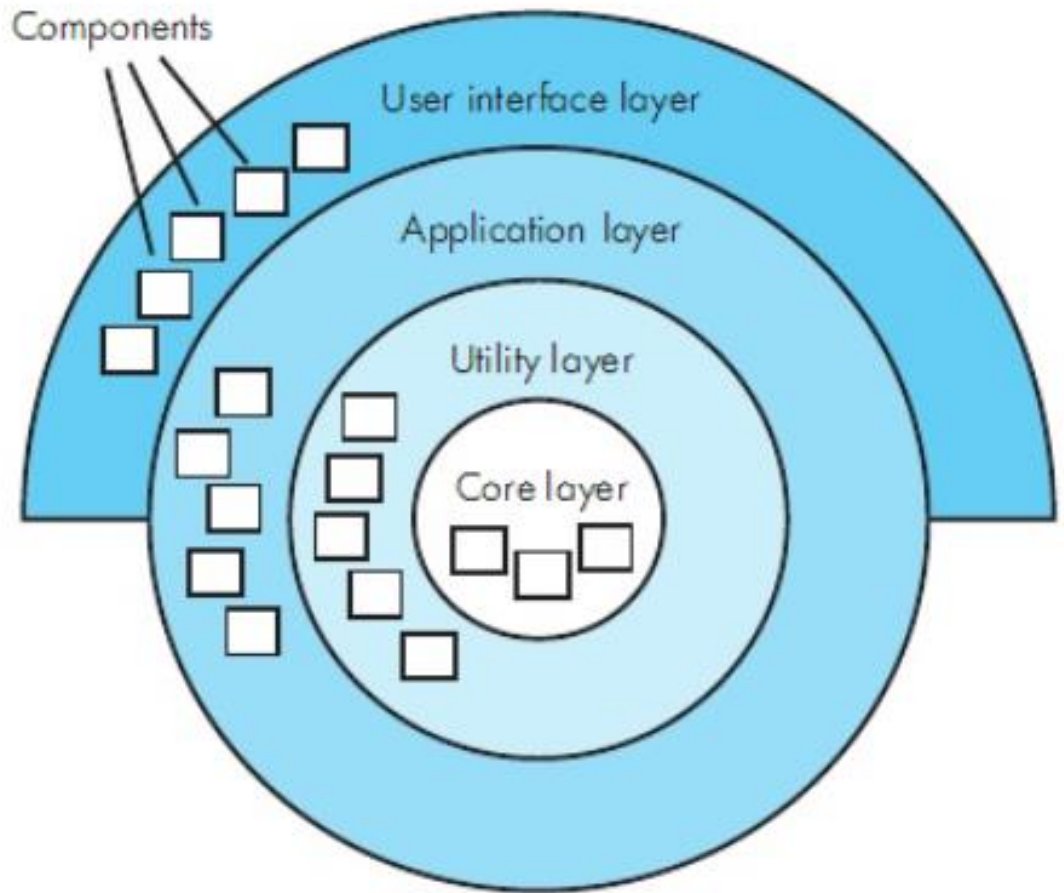
Architectural Styles and Strategies

Layering

- Layers are hierarchical
 - Each layer provides service to the one outside it and acts as a client to the layer inside it
 - Layer bridging: allowing a layer to access the services of layers below its lower neighbor
- The design includes protocols
 - Explain how each pair of layers will interact
- Advantages
 - High levels of abstraction
 - Relatively easy to add and modify a layer
- Disadvantages
 - Not always easy to structure system layers
 - System performance may suffer from the extra coordination among layers

Architectural Styles and Strategies

Example of Layering System



When to Consider This Style: Layered Architecture

- A reason to consider the layered architecture is when a majority of your changes are isolated to specific layers within the application. For example, changes isolated to only business rules that don't impact the user interface, changes involving only the user interface look-and-feel, migration to a new user interface framework, and even migration to a new type of database are all isolated to a specific layer in the architecture, making it easier to isolate the components impacted by the change.
- Because the layered architecture is a technically partitioned architecture, it's a good fit if the team structure is also technically partitioned.

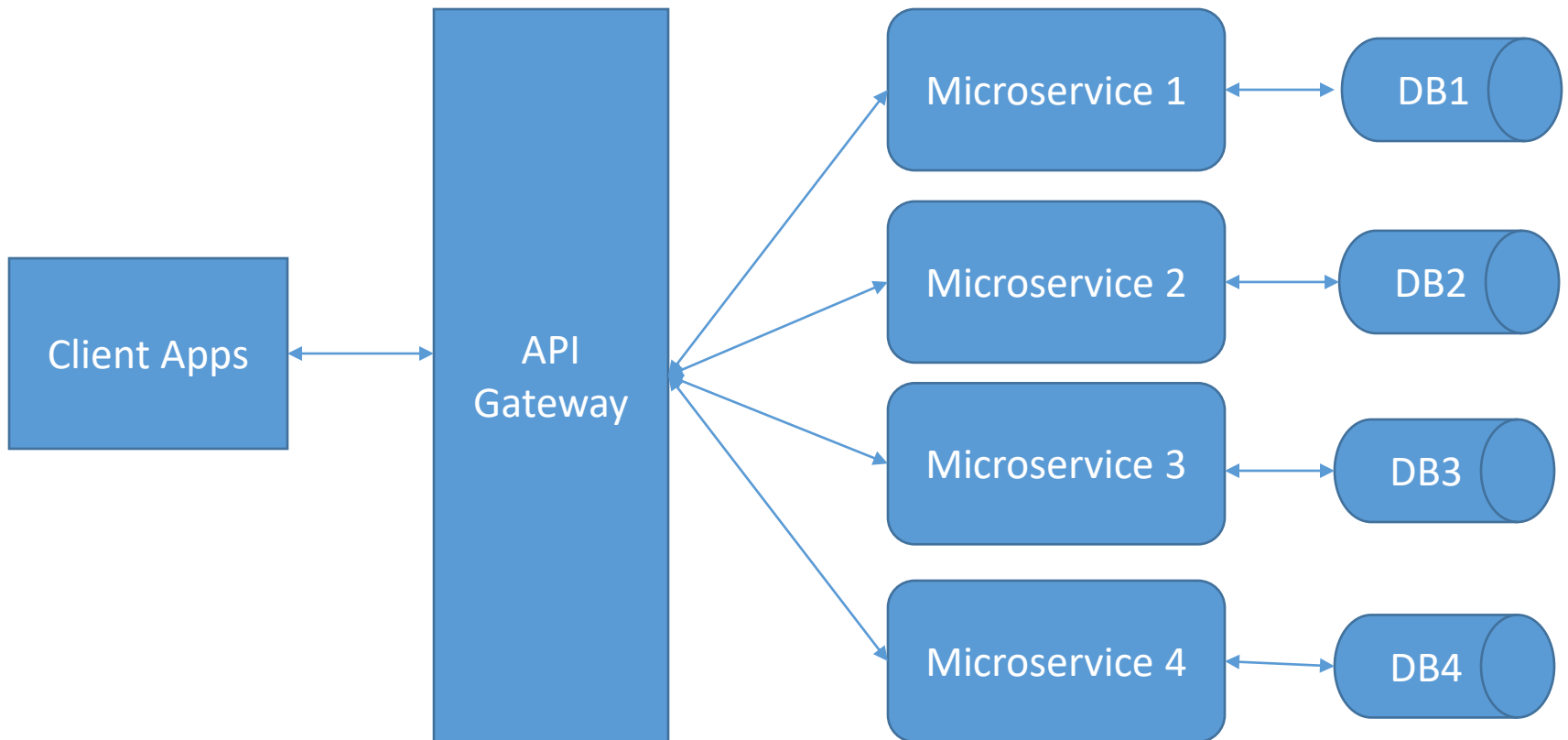
Layered Architecture

- **Use cases:**E-commerce Platform: Separates user interface, business logic, and data storage for efficient management and updates.
- Banking Application: Ensures clear separation between customer interactions, transaction processing, and data storage.
- Content Management System: Segregates content presentation, management, and storage for easier content updates.
- Not to consider the layered architecture is if you have high operational concerns for your application—things like scalability, elasticity, fault tolerance, and performance.

Drawbacks:

- Communication overhead between layers can impact performance.
- Tight coupling risks if layer boundaries aren't well-defined.
- May become overly complex with numerous layers.

Microservices Architecture



Microservices Architecture

- A microservice is defined as a single-purpose, separately deployed unit of software that does one thing really, really well.
- Microservices are meant to represent single-purpose functions
- Services are accessed typically through an API gateway.
- The primary job of the API gateway in microservices is to hide the location and implementation of the corresponding services that correspond to the API gateway endpoints.
- Microservices is the only architecture style that requires data to be broken up and distributed across separate services. The reason for this need is the sheer number of services usually found within a typical microservices architecture

Microservices Architecture

- An interesting use case for microservices is that of business intelligence and analytics reporting. Each report, query, data feed, or data analytics can be developed as a separate microservice, all accessing data within a data lake or data warehouse.

When to Consider This Style: Microservices Architecture

- Is it feasible to break apart your application functionality into dozens or hundreds of separate and distinct pieces of functionality that are independent from each other?
- You should also consider microservices if you have plans for lots of extensibility in your existing architecture. Adding functionality in microservices is sometimes simply a matter of creating a service, wrapping it in a container, creating an API endpoint, and deploying that service.

When Not to Consider This Style:

Microservices Architecture

- If you find that all of that separately deployed functionality needs to be tied together with complex workflows and lots of inter-service communication or orchestration, then this is not an architecture you should consider
- If your data is tightly coupled and monolithic in nature (meaning it's not feasible to break apart your data into several dozen to several hundred separate schemas or databases), then run away in the opposite direction of microservices.
- microservices is perhaps the most complex architecture style that exists today. Consequently, it is also very expensive.

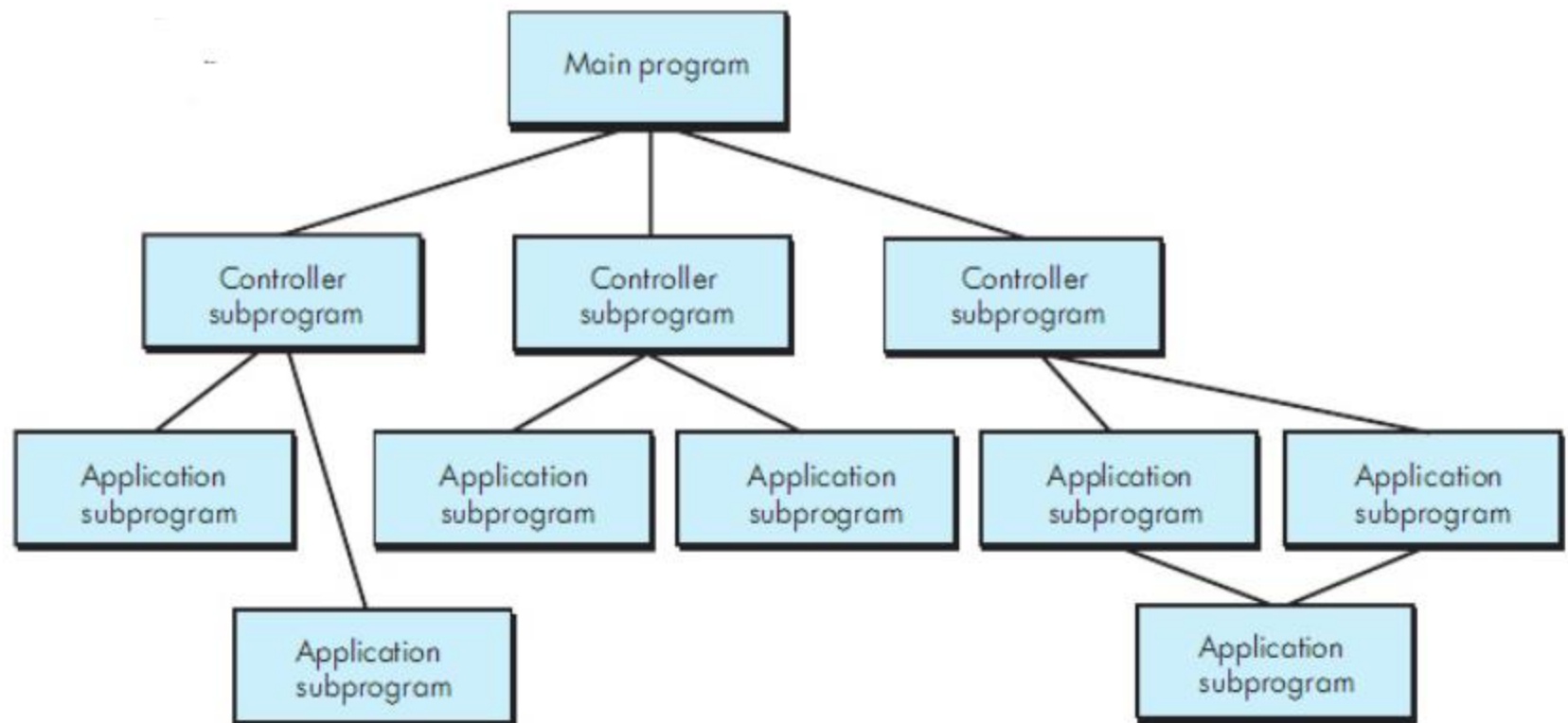
Microservices Architecture

- **Use Cases:** E-commerce Marketplace: Different microservices handle user management, product catalog, payments, and order processing.
- Ride-Sharing Application: Separate services manage user authentication, ride requests, driver tracking, and payments.
- Streaming Platform: Microservices for content delivery, user profiles, recommendations, and billing.
- **Drawbacks:**
 - Complexity in managing distributed architecture.
 - Challenges in ensuring data consistency across services.
 - Communication overhead between services can impact performance.

Architecture Styles and Strategies

Call and Return

- Main program/sub program
- Remote procedure call



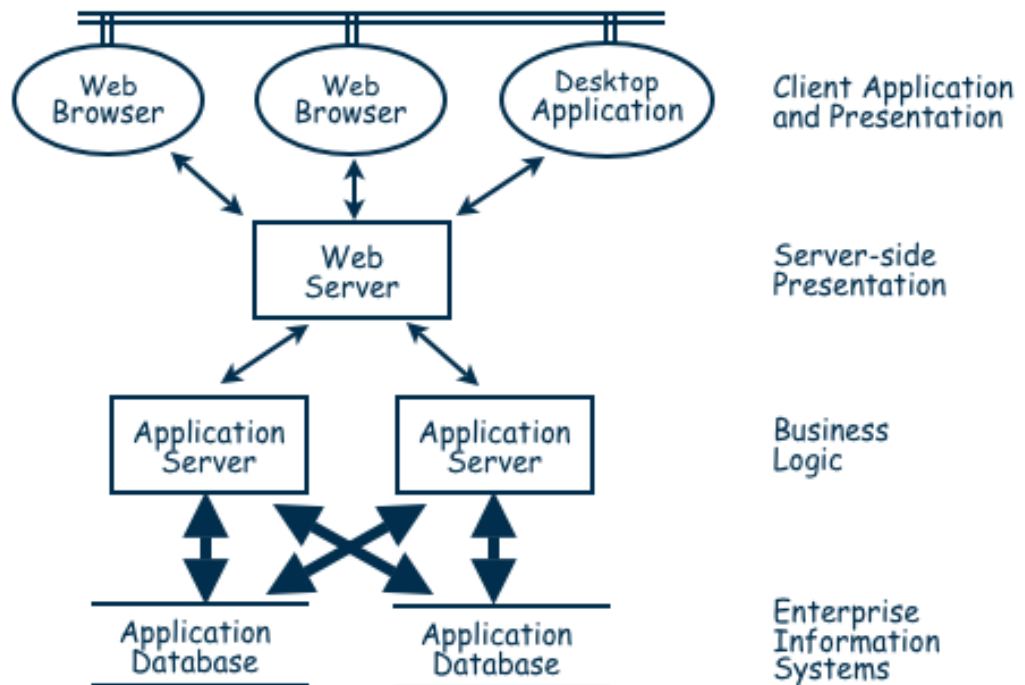
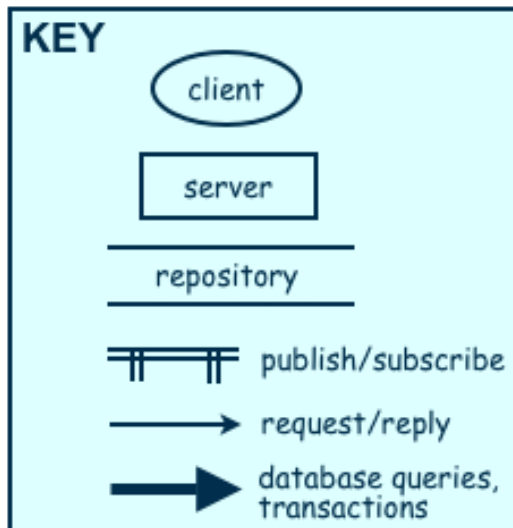
Architectural Styles and Strategies

Combining Architectural Styles

- Actual software architectures rarely based on purely one style
- Architectural styles can be combined in several ways
 - Use different styles at different layers (e.g., overall client-server architecture with server component decomposed into layers)
 - Use mixture of styles to model different components or types of interaction (e.g., client components interact with one another using publish-subscribe communications)
- If architecture is expressed as collection of models, documentation must be created to show relation between models

Architectural Styles and Strategies

Combination of WHICH Architecture Styles?



Combination of Publish-Subscribe, Client-Server, Layering, Repository

Hybrid Architecture

- **Use Cases:**

- Complex Enterprise Systems: Hybrid architectures can balance the strengths of different patterns to meet diverse needs.
- Scalable Web Applications: Combining microservices with event-driven patterns to ensure responsiveness and modularity.
- Real-Time Analytics: Using a combination of event-driven and space-based patterns for efficient data processing.

- **Drawbacks:**

- Complexity in managing hybrid architectures.
- Integration challenges between different patterns.
- Requires careful design and planning to ensure cohesiveness.

Architecture	Description	When to Use
Pipe and Filter	A series of processing elements (filters) connected by pipes that pass data from one to the next. Each filter transforms data before passing it on.	Use when you need sequential data processing, such as in data pipelines, streaming, or batch processing systems.
Client-Server	The client requests services, and the server provides them. Centralized control with distributed clients.	Use when you need centralized control with distributed users, such as in web applications, databases, or cloud services.
Peer-to-Peer (P2P)	Nodes (peers) act both as clients and servers. Each peer can request and provide services.	Use for decentralized systems where scalability and redundancy are important, such as in file-sharing networks (BitTorrent) or blockchain.
Repository	Centralized data storage (repository) with independent components accessing and updating the repository.	Use when multiple independent systems need to share and update the same data, such as in databases or version control systems.
Layered	Organized into layers, where each layer provides services to the layer above it and uses services from the layer below.	Use when you need clear separation of concerns and abstraction, such as in operating systems, network protocols, or enterprise applications.
Microservices	A system composed of small, independent services, each focused on a specific function. Each service can be developed, deployed, and scaled independently.	Use for large, complex, and scalable systems that require flexibility and independent deployment, such as in modern cloud-native applications (e.g., Netflix, Amazon).

References

- SE book by Pfleeger
- SE book by Pressman
- SE book by Ian Sommerville
- UCF slides for SE course (a few slides have been reused)